

Объектно-ориентированное программирование

Лекция 9: Обзор паттернов и Фундаментальные паттерны

Классификация паттернов GoF. Преимущества и недостатки. Интерфейс (PEP 544) и его реализация через Protocol. Делегирование (`__getattr__` , `__setattr__`).
Контейнер свойств.

- Описание взаимодействия объектов и классов, адаптированных для решения общей задачи проектирования в конкретном контексте.
- Повторяемая архитектурная конструкция в сфере проектирования программного обеспечения, предлагающая решение проблемы проектирования в рамках некоторого часто возникающего контекста.

1. Имя
2. Решаемая задача
3. Предлагаемое решение
4. Ожидаемый результат, последствия

- Ссылаясь на него, можно сразу описать проблему проектирования; ее решения и их последствия.
- Присваивание паттернам имен позволяет проектировать на более высоком уровне абстракции.
- С помощью словаря паттернов можно вести обсуждение с коллегами, упоминать паттерны в документации, в тонкостях представлять дизайн системы.

- Описание того, когда следует применять паттерн.
- Необходимо сформулировать задачу и ее контекст. Может описываться конкретная проблема проектирования, например способ представления алгоритмов в виде объектов.
- Иногда отмечается, какие структуры классов или объектов свидетельствуют о негибком дизайне. Также может включаться перечень условий, при выполнении которых имеет смысл применять данный паттерн.

- Описание элементов дизайна, отношений между ними, функций каждого элемента.
- Конкретный дизайн или реализация не имеются в виду, поскольку паттерн - это шаблон, применимый в самых разных ситуациях.
- Наоборот, дается абстрактное описание задачи проектирования и того, как она может быть решена с помощью некоего весьма обобщенного сочетания элементов (классов, объектов, свойств, методов).

- Следствия применения паттерна и разного рода компромиссы.
- Хотя при описании проектных решений о последствиях часто не упоминают, знать о них необходимо, чтобы можно было выбрать между различными вариантами и оценить преимущества и недостатки данного паттерна.
- Здесь речь идет и о выборе языка и реализации. Поскольку в объектно-ориентированном проектировании повторное использование зачастую является важным фактором, то к результатам следует относить и влияние на степень гибкости, расширяемости и переносимости системы.
- Перечисление всех последствий поможет вам понять и оценить их роль.

Паттерны бывают и другими

- Архитектурные шаблоны - это паттерны высокоуровневого проектирования системы
- Алгоритмы - это вычислительные паттерны



Университет
Сириус
Колледж

Виды паттернов проектирования



Университет
Сириус
Колледж

Виды паттернов проектирования

Фундаментальные

Фундаментальные

- Интерфейс

Фундаментальные

- Интерфейс
- Делегирование

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер
свойств

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер
свойств
- Канал событий

Фундаментальные Порождающие

- Интерфейс
- Делегирование
- Контейнер
свойств
- Канал событий

Фундаментальные	Порождающие
-----------------	-------------

- | | |
|---|---|
| <ul style="list-style-type: none">■ Интерфейс■ Делегирование■ Контейнер свойств■ Канал событий | <ul style="list-style-type: none">■ Абстрактная фабрика |
|---|---|



Фундаментальные	Порождающие
-----------------	-------------

- | | |
|-----------------|---------------|
| ■ Интерфейс | ■ Абстрактная |
| ■ Делегирование | фабрика |
| ■ Контейнер | ■ Строитель |
| свойств | |
| ■ Канал событий | |

Фундаментальные

Порождающие

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

- Абстрактная фабрика
- Строитель
- Фабричный метод

Фундаментальные

Порождающие

- | | |
|-----------------|-------------------|
| ■ Интерфейс | ■ Абстрактная |
| ■ Делегирование | фабрика |
| ■ Контейнер | ■ Строитель |
| свойств | ■ Фабричный метод |
| ■ Канал событий | ■ Прототип |

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

- Адаптер

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

- Адаптер
- Фасад

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

- Адаптер
- Фасад
- Мост

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

- Адаптер
- Фасад
- Мост
- Компоновщик

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

- Адаптер
- Фасад
- Мост
- Компоновщик
- Декоратор

Фундаментальные

- Интерфейс
- Делегирование
- Контейнер свойств
- Канал событий

Порождающие

- Абстрактная фабрика
- Строитель
- Фабричный метод
- Прототип
- Синглтон

Структурные

- Адаптер
- Фасад
- Мост
- Компоновщик
- Декоратор
- Приспособленец

Фундаментальные	Порождающие	Структурные
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник ■ Хранитель

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник ■ Хранитель ■ Наблюдатель

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник ■ Хранитель ■ Наблюдатель ■ Состояние

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник ■ Хранитель ■ Наблюдатель ■ Состояние ■ Стратегия

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник ■ Хранитель ■ Наблюдатель ■ Состояние ■ Стратегия ■ Шаблонный метод

Фундаментальные	Порождающие	Структурные	Поведенческие
■ Интерфейс ■ Делегирование ■ Контейнер свойств ■ Канал событий	■ Абстрактная фабрика ■ Строитель ■ Фабричный метод ■ Прототип ■ Синглтон	■ Адаптер ■ Фасад ■ Мост ■ Компоновщик ■ Декоратор ■ Приспособленец ■ Заместитель	■ Цепочка обязанностей ■ Команда ■ Интерпретатор ■ Итератор ■ Посредник ■ Хранитель ■ Наблюдатель ■ Состояние ■ Стратегия ■ Шаблонный метод ■ Посетитель

Фундаментальные (основные, базовые)

- Это паттерны проектирования, которые, скорее, являются основами, на которых строится всё остальное взаимодействие.
- Фундаментальные паттерны в том или ином виде реализованы в объектно-ориентированных языках программирования.

- Интерфейс - общий метод или общий класс.
- Неизменяемый интерфейс - метод, создающий неизменяемый объект.
- Интерфейс-маркер - метод с проверкой типов.

```
1 class InformalParserInterface:
2     def load_data_source(self, path: str, file_name: str) -> str:
3         """Load in the file for extracting text."""
4         pass
5
6     def extract_text(self, full_file_name: str) -> dict:
7         """Extract text from the currently loaded file."""
8         pass
9
10
11 class PdfParser(InformalParserInterface):
12     """Extract text from a PDF"""
13     def load_data_source(self, path: str, file_name: str) -> str:
14         """Overrides InformalParserInterface.load_data_source()"""
15         pass
```

```
1 class InformalParserInterface:
2     def load_data_source(self, path: str, file_name: str) -> str:
3         """Load in the file for extracting text."""
4         pass
5
6     def extract_text(self, full_file_name: str) -> dict:
7         """Extract text from the currently loaded file."""
8         pass
9
10
11 class PdfParser(InformalParserInterface):
12     """Extract text from a PDF"""
13     def load_data_source(self, path: str, file_name: str) -> str:
14         """Overrides InformalParserInterface.load_data_source()"""
15         pass
```

```
8         pass
9
10
11 class PdfParser(InformalParserInterface):
12     """Extract text from a PDF"""
13     def load_data_source(self, path: str, file_name: str) -> str:
14         """Overrides InformalParserInterface.load_data_source()"""
15         pass
16
17     def extract_text(self, full_file_path: str) -> dict:
18         """Overrides InformalParserInterface.extract_text()"""
19         pass
20
21
22 class EmlParser(InformalParserInterface):
```

```
20
21
22 class EmlParser(InformalParserInterface):
23     """Extract text from an email"""
24     def load_data_source(self, path: str, file_name: str) -> str:
25         """Overrides InformalParserInterface.load_data_source()"""
26         pass
27
28     def extract_text_from_email(self, full_file_path: str) -> dict:
29         """A method defined only in EmlParser.
30         Does not override InformalParserInterface.extract_text()
31         """
32         pass
33
34
```

```
23     """Extract text from an email"""
24     def load_data_source(self, path: str, file_name: str) -> str:
25         """Overrides InformalParserInterface.load_data_source()"""
26         pass
27
28     def extract_text_from_email(self, full_file_path: str) -> dict:
29         """A method defined only in EmlParser.
30         Does not override InformalParserInterface.extract_text()
31         """
32         pass
33
34
35 issubclass(PdfParser, InformalParserInterface) # True
36 issubclass(EmlParser, InformalParserInterface) # True
37
```



```
28     def extract_text_from_email(self, full_file_path: str) -> dict:
29         """A method defined only in EmlParser.
30         Does not override InformalParserInterface.extract_text()
31         """
32         pass
33
34
35 subclass(PdfParser, InformalParserInterface) # True
36 subclass(EmlParser, InformalParserInterface) # True
37
38 PdfParser.__mro__
39 # (__main__.PdfParser, __main__.InformalParserInterface, object)
40
41 EmlParser.__mro__
42 # (__main__.EmlParser, __main__.InformalParserInterface, object)
```




```
28     def extract_text_from_email(self, full_file_path: str) -> dict:
29         """A method defined only in EmlParser.
30         Does not override InformalParserInterface.extract_text()
31         """
32         pass
33
34
35 issubclass(PdfParser, InformalParserInterface) # True
36 issubclass(EmlParser, InformalParserInterface) # True
37
38 PdfParser.__mro__
39 # (__main__.PdfParser, __main__.InformalParserInterface, object)
40
41 EmlParser.__mro__
42 # (__main__.EmlParser, __main__.InformalParserInterface, object)
```

```
1 class ParserMeta(type):
2     """A Parser metaclass that will be used for parser class creation.
3     """
4     def __instancecheck__(cls, instance):
5         return cls.__subclasscheck__(type(instance))
6
7     def __subclasscheck__(cls, subclass):
8         return (hasattr(subclass, 'load_data_source') and
9                 callable(subclass.load_data_source) and
10                hasattr(subclass, 'extract_text') and
11                callable(subclass.extract_text))
12
13
14 class UpdatedInformalParserInterface(metaclass=ParserMeta):
15     """This interface is used for concrete classes to inherit from
```

```
1 class ParserMeta(type):
2     """A Parser metaclass that will be used for parser class creation.
3     """
4     def __instancecheck__(cls, instance):
5         return cls.__subclasscheck__(type(instance))
6
7     def __subclasscheck__(cls, subclass):
8         return (hasattr(subclass, 'load_data_source') and
9                 callable(subclass.load_data_source) and
10                hasattr(subclass, 'extract_text') and
11                callable(subclass.extract_text))
12
13
14 class UpdatedInformalParserInterface(metaclass=ParserMeta):
15     """This interface is used for concrete classes to inherit from
```

```
1 class ParserMeta(type):
2     """A Parser metaclass that will be used for parser class creation.
3     """
4     def __instancecheck__(cls, instance):
5         return cls.__subclasscheck__(type(instance))
6
7     def __subclasscheck__(cls, subclass):
8         return (hasattr(subclass, 'load_data_source') and
9                 callable(subclass.load_data_source) and
10                hasattr(subclass, 'extract_text') and
11                callable(subclass.extract_text))
12
13
14 class UpdatedInformalParserInterface(metaclass=ParserMeta):
15     """This interface is used for concrete classes to inherit from
```

```
2     """A Parser metaclass that will be used for parser class creation.
3     """
4     def __instancecheck__(cls, instance):
5         return cls.__subclasscheck__(type(instance))
6
7     def __subclasscheck__(cls, subclass):
8         return (hasattr(subclass, 'load_data_source') and
9                 callable(subclass.load_data_source) and
10                hasattr(subclass, 'extract_text') and
11                callable(subclass.extract_text))
12
13
14 class UpdatedInformalParserInterface(metaclass=ParserMeta):
15     """This interface is used for concrete classes to inherit from.
16     There is no need to define the ParserMeta methods as any class
```

```
10         hasattr(subclass, 'extract_text') and
11         callable(subclass.extract_text))
12
13
14 class UpdatedInformalParserInterface(metaclass=ParserMeta):
15     """This interface is used for concrete classes to inherit from.
16     There is no need to define the ParserMeta methods as any class
17     as they are implicitly made available via .__subclasscheck__().
18     """
19     pass
20
21
22 class PdfParserNew:
23     """Extract text from a PDF."""
24     def load_data_source(self, path: str, file name: str) -> str:
```

```
19     pass
20
21
22 class PdfParserNew:
23     """Extract text from a PDF."""
24     def load_data_source(self, path: str, file_name: str) -> str:
25         """Overrides UpdatedInformalParserInterface.load_data_source()"""
26         pass
27
28     def extract_text(self, full_file_path: str) -> dict:
29         """Overrides UpdatedInformalParserInterface.extract_text()"""
30         pass
31
32
33 class EmlParserNew:
```

```
31
32
33 class EmlParserNew:
34     """Extract text from an email."""
35     def load_data_source(self, path: str, file_name: str) -> str:
36         """Overrides UpdatedInformalParserInterface.load_data_source()"""
37         pass
38
39     def extract_text_from_email(self, full_file_path: str) -> dict:
40         """A method defined only in EmlParser.
41         Does not override UpdatedInformalParserInterface.extract_text()
42         """
43         pass
44
45
```




```
36         overrides UpdatedInformalParserInterface.load_data_source()
37     pass
38
39     def extract_text_from_email(self, full_file_path: str) -> dict:
40         """A method defined only in EmlParser.
41         Does not override UpdatedInformalParserInterface.extract_text()
42         """
43     pass
44
45
46 issubclass(PdfParserNew, UpdatedInformalParserInterface) # True
47 issubclass(EmlParserNew, UpdatedInformalParserInterface) # False
48
49 PdfParserNew.__mro__
50 # (<class '__main__.PdfParserNew'>, <class 'object'>)
```

```
36         overrides UpdatedInformalParserInterface.load_data_source()
37     pass
38
39     def extract_text_from_email(self, full_file_path: str) -> dict:
40         """A method defined only in EmlParser.
41         Does not override UpdatedInformalParserInterface.extract_text()
42         """
43     pass
44
45
46 issubclass(PdfParserNew, UpdatedInformalParserInterface) # True
47 issubclass(EmlParserNew, UpdatedInformalParserInterface) # False
48
49 PdfParserNew.__mro__
50 # (<class '__main__.PdfParserNew'>, <class 'object'>)
```



```
1  from typing import Protocol
2
3  class IParser(Protocol):
4      def load_data(self, path: str) -> str: ...
5      def extract(self, data: str) -> dict: ...
6
7  class PdfParser: # Не нужно наследовать явно!
8      def load_data(self, path: str) -> str: return "pdf data"
9      def extract(self, data: str) -> dict: return {"text": "..."}
10
11 def process(parser: IParser):
12     parser.load_data("file.pdf")
13
14 process(PdfParser())
```

Функциональный дизайн гарантирует, что каждый модуль компьютерной программы имеет только одну обязанность и исполняет её с минимумом побочных эффектов на другие части программы.

Делегирование - шаблон проектирования, в котором объект внешне выражает некоторое поведение, но в реальности передаёт ответственность за выполнение этого поведения связанному объекту.

Шаблон делегирования является фундаментальной абстракцией, на основе которой реализованы другие шаблоны - композиция, агрегация, миксины (mixins) и аспекты (aspects).



Фундаментальные паттерны: Делегирование

```
1 class Worker:
2     def work(self):
3         print("Работаю...")
4
5 class Manager:
6     def __init__(self, worker: Worker):
7         self.worker = worker # Агрегация
8
9     def work(self):
10        print("Менеджер поручает задачу:")
11        self.worker.work() # Делегирование
12
13 m = Manager(Worker())
14 m.work()
```

Контейнер свойств предоставляет механизм для динамического расширения объектов дополнительными атрибутами во время выполнения.

Кроме этого, приложению могут потребоваться ещё модули, которые явным образом используют преимущества нового свойства, если оно было добавлено.

В Python есть встроенный метод-пример этого паттерна, общий для всех объектов - `setattr`

```
1 class PropertyContainer:
2     def __init__(self):
3         self._properties = {}
4
5     def add_property(self, key, value):
6         self._properties[key] = value
7
8     def get_property(self, key):
9         return self._properties.get(key)
10
11 # Пример: Персонаж игры с динамическими эффектами
12 hero = PropertyContainer()
13 hero.add_property("invisibility", True)
14 hero.add_property("bonus_damage", 50)
```




```
1 class SmartRecord:
2     def __init__(self):
3         self._data = {}
4
5     def __getattr__(self, name):
6         # Вызывается, если атрибут не найден обычным способом
7         return self._data.get(name)
8
9     def __setattr__(self, name, value):
10        if name == "_data":
11            super().__setattr__(name, value)
12        else:
13            self._data[name] = value
14
15 record = SmartRecord()
```



```
1 class SmartRecord:
2     def __init__(self):
3         self._data = {}
4
5     def __getattr__(self, name):
6         # Вызывается, если атрибут не найден обычным способом
7         return self._data.get(name)
8
9     def __setattr__(self, name, value):
10        if name == "_data":
11            super().__setattr__(name, value)
12        else:
13            self._data[name] = value
14
15 record = SmartRecord()
```



```
1 class SmartRecord:
2     def __init__(self):
3         self._data = {}
4
5     def __getattr__(self, name):
6         # Вызывается, если атрибут не найден обычным способом
7         return self._data.get(name)
8
9     def __setattr__(self, name, value):
10        if name == "_data":
11            super().__setattr__(name, value)
12        else:
13            self._data[name] = value
14
15 record = SmartRecord()
```



Фундаментальные паттерны: Контейнер свойств

```
4
5     def __getattr__(self, name):
6         # Вызывается, если атрибут не найден обычным способом
7         return self._data.get(name)
8
9     def __setattr__(self, name, value):
10        if name == "_data":
11            super().__setattr__(name, value)
12        else:
13            self._data[name] = value
14
15 record = SmartRecord()
16 record.title = "Отчет №1" # Динамическое создание свойства
17 record.author = "Иванов"
18
```



```
0         # вызывается, если атрибут не найден обычным способом
7         return self._data.get(name)
8
9     def __setattr__(self, name, value):
10         if name == "_data":
11             super().__setattr__(name, value)
12         else:
13             self._data[name] = value
14
15 record = SmartRecord()
16 record.title = "Отчет №1" # Динамическое создание свойства
17 record.author = "Иванов"
18
19 print(record.title)      # Отчет №1
20 print(record._data)      # {'title': 'Отчет №1', 'author': 'Иванов'}
```

Используется для создания канала связи и коммуникации через него посредством событий. Этот канал обеспечивает возможность разным издателям публиковать события и подписчикам, подписываясь на них, получать уведомления.

В python типичным примером `Event` и `Condition` в конкурентном программировании, где одни абстракции могут подписываться на уведомления (`wait`), а другие - публиковать их (`notify`).



```
1 class EventChannel:
2     def __init__(self):
3         self.subscribers = []
4
5     def subscribe(self, func):
6         self.subscribers.append(func)
7
8     def publish(self, data):
9         for func in self.subscribers:
10             func(data)
11
12 channel = EventChannel()
13 channel.subscribe(lambda d: print(f"Логгер: {d}"))
14 channel.subscribe(lambda d: print(f"UI: Обновляю экран ({d})"))
15 channel.publish("Пользователь вошел")
```